

Ciencia de Datos Aplicada a la Minería

Manejo y preprocesamiento de datos

Sesión 5 *Preprocesamiento de datos I*

Instructor:

➤ *Ph.D. (c) Jorge Lliguizaca Dávila*



Contenido



Limpieza de Datos



Tratamiento de Datos Nulos



Tratamiento de Datos Atípicos (Outliers)



Escalado y Normalización de Características

Limpieza de Datos



Problemas comunes de datos

- Restricciones de tipos de datos
- Restricciones de rangos de datos
- Restricciones de unicidad



Problemas de datos categóricos y texto

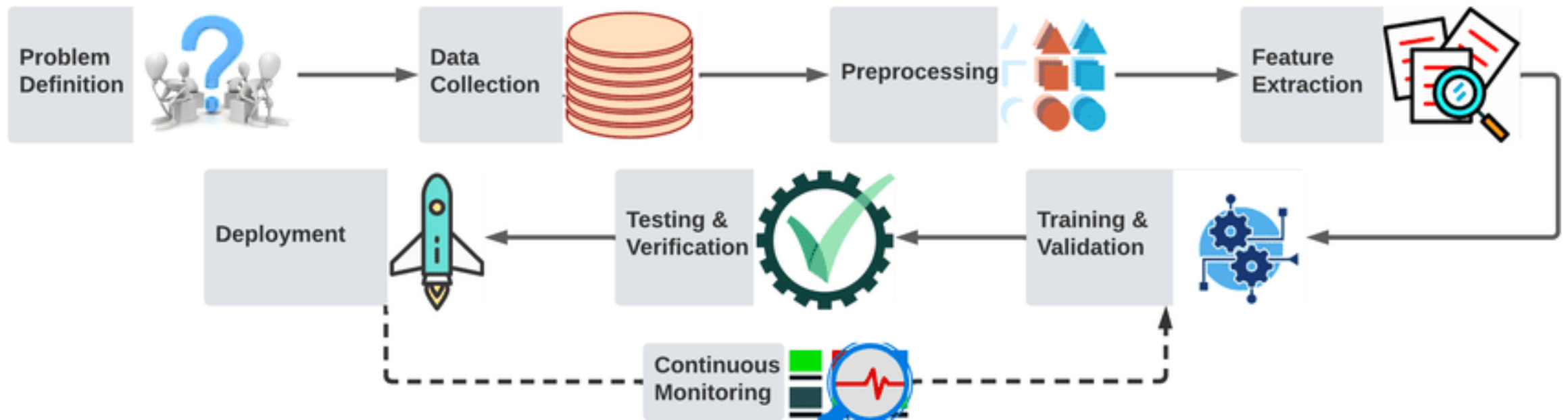
- Restricciones de categorías
- Limpieza de datos de texto



Problemas avanzados de datos

- Uniformidad (homogeneidad)
- Validación cruzada
- Integridad (Compleitud)

Machine Learning Pipeline



Problemas Avanzados de Datos – Uniformidad

Column	Unit
Temperature	32 °C is also 89.6 °F
Weight	70 Kg is also 11 st.
Date	26-11-2019 is also 26, November, 2019
Money	100\$ is also 10763.90¥

Problemas Avanzados de Datos – Uniformidad

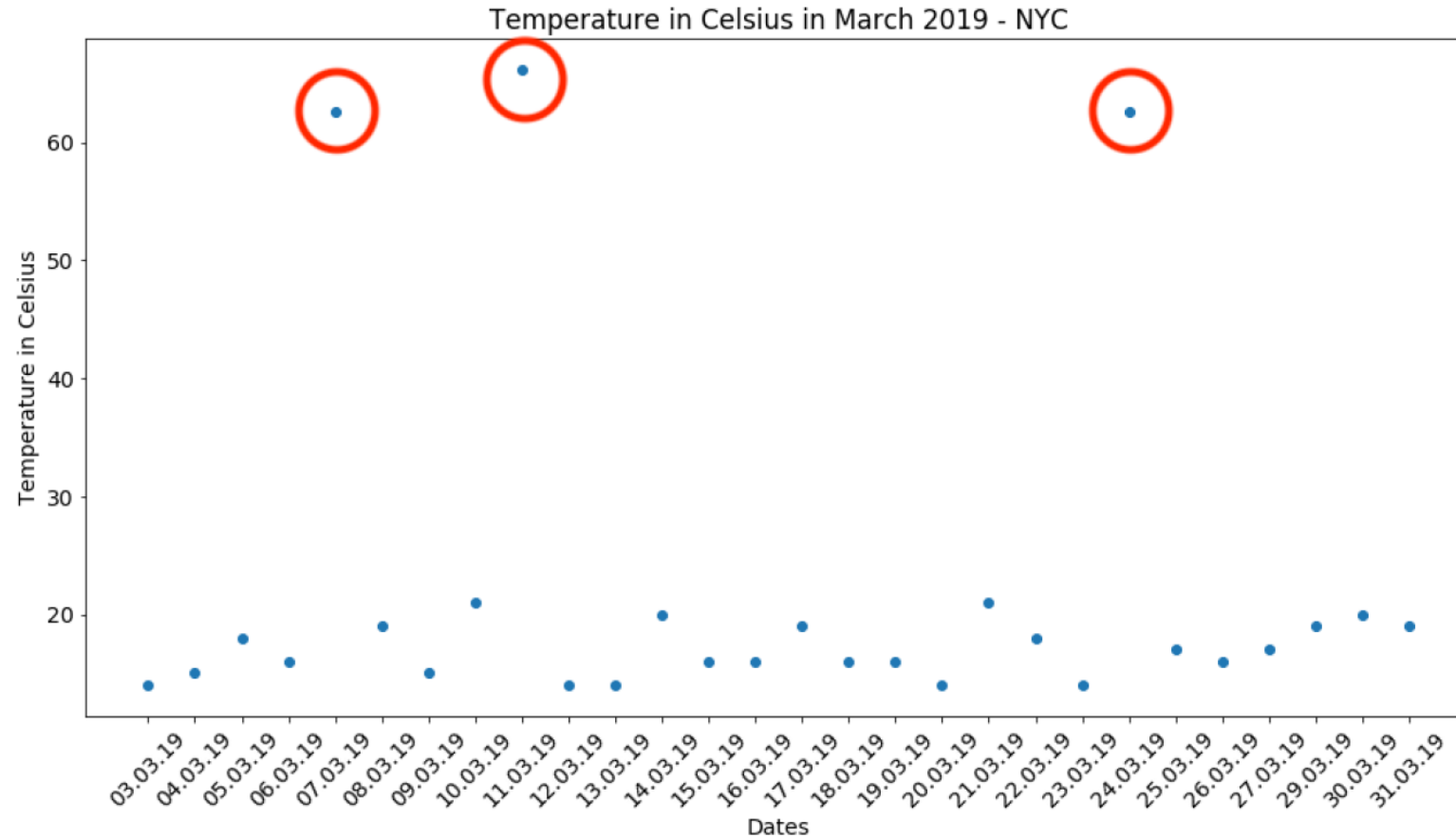
Ejemplo:

```
temperatures = pd.read_csv('temperature.csv')  
temperatures.head()
```

	Date	Temperature
0	03.03.19	14.0
1	04.03.19	15.0
2	05.03.19	18.0
3	06.03.19	16.0
4	07.03.19	62.6

Problemas Avanzados de Datos - Uniformidad

Ejemplo:



Problemas Avanzados de Datos – Uniformidad

Tratar columna Temperatura:

$$C = (F - 32) \times \frac{5}{9}$$

```
temp_fah = temperatures.loc[temperatures['Temperature'] > 40, 'Temperature']  
temp_cels = (temp_fah - 32) * (5/9)  
temperatures.loc[temperatures['Temperature'] > 40, 'Temperature'] = temp_cels
```


Problemas Avanzados de Datos - Uniformidad

Tratar Datos de Tipo Fecha:

```
birthdays.head()
```

	Birthday	First name	Last name
0	27/27/19	Rowan	Nunez
1	03-29-19	Brynn	Yang
2	March 3rd, 2019	Sophia	Reilly
3	24-03-19	Deacon	Prince
4	06-03-19	Griffith	Neal

Problemas Avanzados de Datos - Uniformidad

Tratar Datos de Tipo Fecha:

```
birthdays.head()
```

	Birthday	First name	Last name
0	27/27/19	Rowan	Nunez
1	03-29-19	Brynn	Yang
2	March 3rd, 2019	Sophia	Reilly
3	24-03-19	Deacon	Prince
4	06-03-19	Griffith	Neal

??

MM-DD-YY

Month D, YYYY

Problemas Avanzados de Datos – Uniformidad

Formateo datetime:

Date	datetime format
25-12-2019	%d-%m-%Y
December 25th 2019	%c
12-25-2019	%m-%d-%Y
...	...

Método Pandas.to_datetime()

- Pueden reconocer la mayoría de los formatos de forma automática.
- Algunas veces puede fallar con formatos erróneos o irreconocibles.

Problemas Avanzados de Datos – Uniformidad

Formate datetime:

```
# Converts to datetime – but won't work!  
birthdays['Birthday'] = pd.to_datetime(birthdays['Birthday'])
```

```
ValueError: month must be in 1..12
```

```
# Will work!  
birthdays['Birthday'] = pd.to_datetime(birthdays['Birthday'],  
                                       # Attempt to infer format of each date  
                                       infer_datetime_format=True,  
                                       # Return NA for rows where conversion failed  
                                       errors = 'coerce')
```

Problemas Avanzados de Datos - Uniformidad

Formateo datetime:

¡Data tratada!

```
birthdays.head()
```

```
   Birthday First name Last name
0         NaT      Rowan   Nunez
1 2019-03-29    Brynn    Yang
2 2019-03-03    Sophia  Reilly
3 2019-03-24    Deacon  Prince
4 2019-06-03  Griffith    Neal
```

Problemas Avanzados de Datos – Uniformidad

Formateo datetime:

```
birthdays['Birthday'] = birthdays['Birthday'].dt.strftime("%d-%m-%Y")  
birthdays.head()
```

	Birthday	First name	Last name
0	NaT	Rowan	Nunez
1	29-03-2019	Brynn	Yang
2	03-03-2019	Sophia	Reilly
3	24-03-2019	Deacon	Prince
4	03-06-2019	Griffith	Neal

Problemas Avanzados de Datos – Uniformidad

Tratamiento de datos tipo fecha ambiguos

- **¿2019-03-08 es agosto o marzo?**
 - Convertir a NA y tratar los datos de forma adecuada.
 - Inferir formato identificando la fuente de los datos.
 - Inferir formato identificando datos previos y posteriores dentro del DataFrame.

Problemas Avanzados de Datos – Validación Cruzada

- El uso de **múltiples** campos en un conjunto de datos para comprobar la **integridad de los datos**.

	flight_number	economy_class	business_class	first_class	total_passengers
0	DL140	100	60	40	200
1	BA248	130	100	70	300
2	MEA124	100	50	50	200
3	AFR939	140	70	90	300
4	TKA101	130	100	20	250

	flight_number	economy_class	business_class	first_class	total_passengers
0	DL140	100	+ 60	+ 40	= 200
1	BA248	130	+ 100	+ 70	= 300
2	MEA124	100	+ 50	+ 50	= 200
3	AFR939	140	+ 70	+ 90	= 300
4	TKA101	130	+ 100	+ 20	= 250

Problemas Avanzados de Datos – Validación Cruzada

- El uso de **múltiples** campos en un conjunto de datos para comprobar la **integridad de los datos**.

flight_number		economy_class	business_class	first_class	total_passengers	
0	DL140	100	+	60	+	200
1	BA248	130	+	100	+	300
2	MEA124	100	+	50	+	200
3	AFR939	140	+	70	+	300
4	TKA101	130	+	100	+	250

```
sum_classes = flights[['economy_class', 'business_class', 'first_class']].sum(axis = 1)
passenger_equ = sum_classes == flights['total_passengers']
# Find and filter out rows with inconsistent passenger totals
inconsistent_pass = flights[~passenger_equ]
consistent_pass = flights[passenger_equ]
```

Problemas Avanzados de Datos – Validación Cruzada

```
users.head()
```

	user_id	Age	Birthday
0	32985	22	1998-03-02
1	94387	27	1993-12-04
2	34236	42	1978-11-24
3	12551	31	1989-01-03
4	55212	18	2002-07-02

```
import pandas as pd
import datetime as dt

# Convert to datetime and get today's date
users['Birthday'] = pd.to_datetime(users['Birthday'])
today = dt.date.today()

# For each row in the Birthday column, calculate year difference
age_manual = today.year - users['Birthday'].dt.year

# Find instances where ages match
age_equ = age_manual == users['Age']

# Find and filter out rows with inconsistent age
inconsistent_age = users[~age_equ]
consistent_age = users[age_equ]
```

Problemas Avanzados de Datos – Validación Cruzada

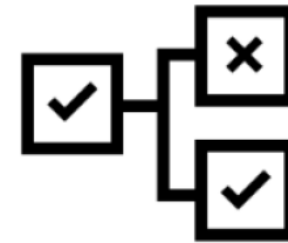
¿Qué hacer cuando se detentan inconsistencias?



*Dropping
Data*



*Set to missing
and impute*



*Apply rules from
domain knowledge*

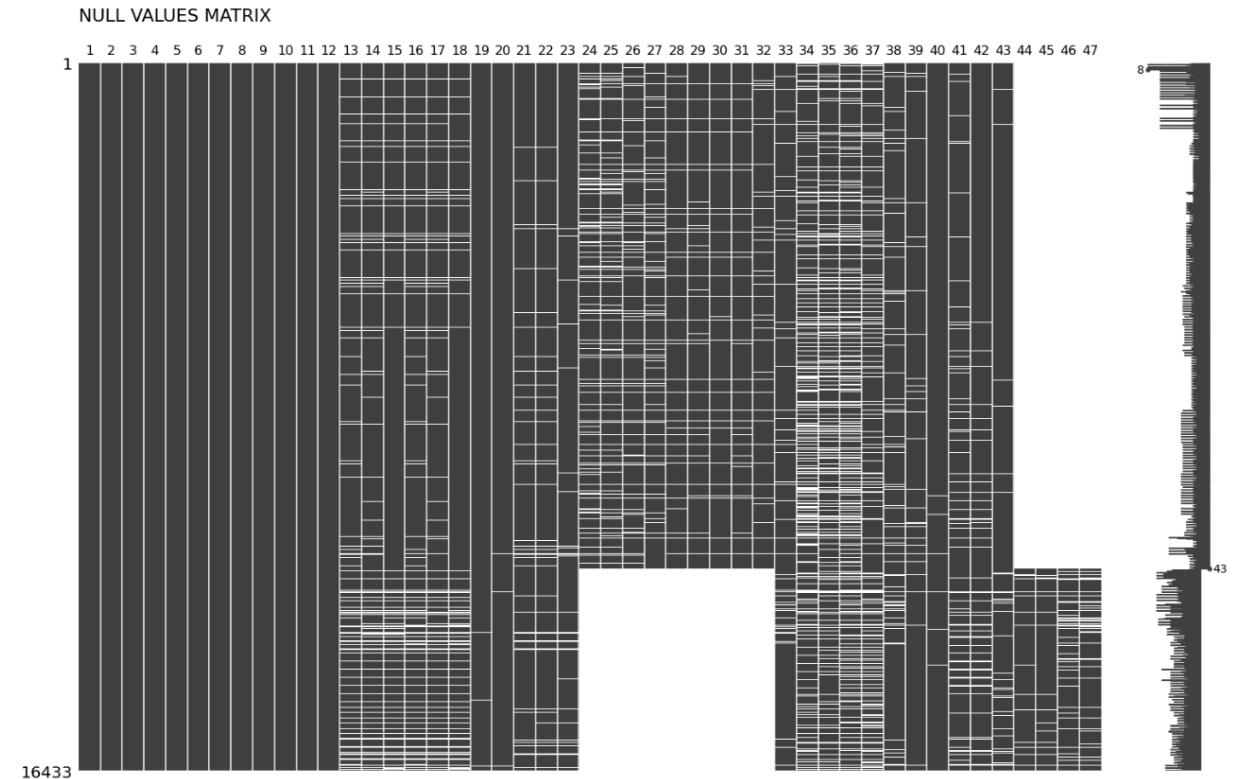
Problemas Avanzados de Datos – Integridad (Compleitud)

¿Qué son los datos nulos?

- Ocurren cuando no se almacena ningún valor de datos para una variable en una observación.
- Pueden ser representados como: **NA**, **nan**, **o**, **.**

¿Cómo pueden ocurrir?

- Errores técnicos
- Errores humanos



Problemas Avanzados de Datos – Integridad (Compleitud)

Ejemplo: Calidad del aire. Inspección de datos nulos

```
import pandas as pd
airquality = pd.read_csv('airquality.csv')
print(airquality)
```

	Date	Temperature	C02	
987	20/04/2004	16.8	0.0	
2119	07/06/2004	18.7	0.8	
2451	20/06/2004	-40.0	NaN	<--
1984	01/06/2004	19.6	1.8	
8299	19/02/2005	11.2	1.2	
...	...	...	...	

```
# Return missing values
airquality.isna()
```

	Date	Temperature	C02
987	False	False	False
2119	False	False	False
2451	False	False	True
1984	False	False	False
8299	False	False	False

```
# Get summary of missingness
airquality.isna().sum()
```

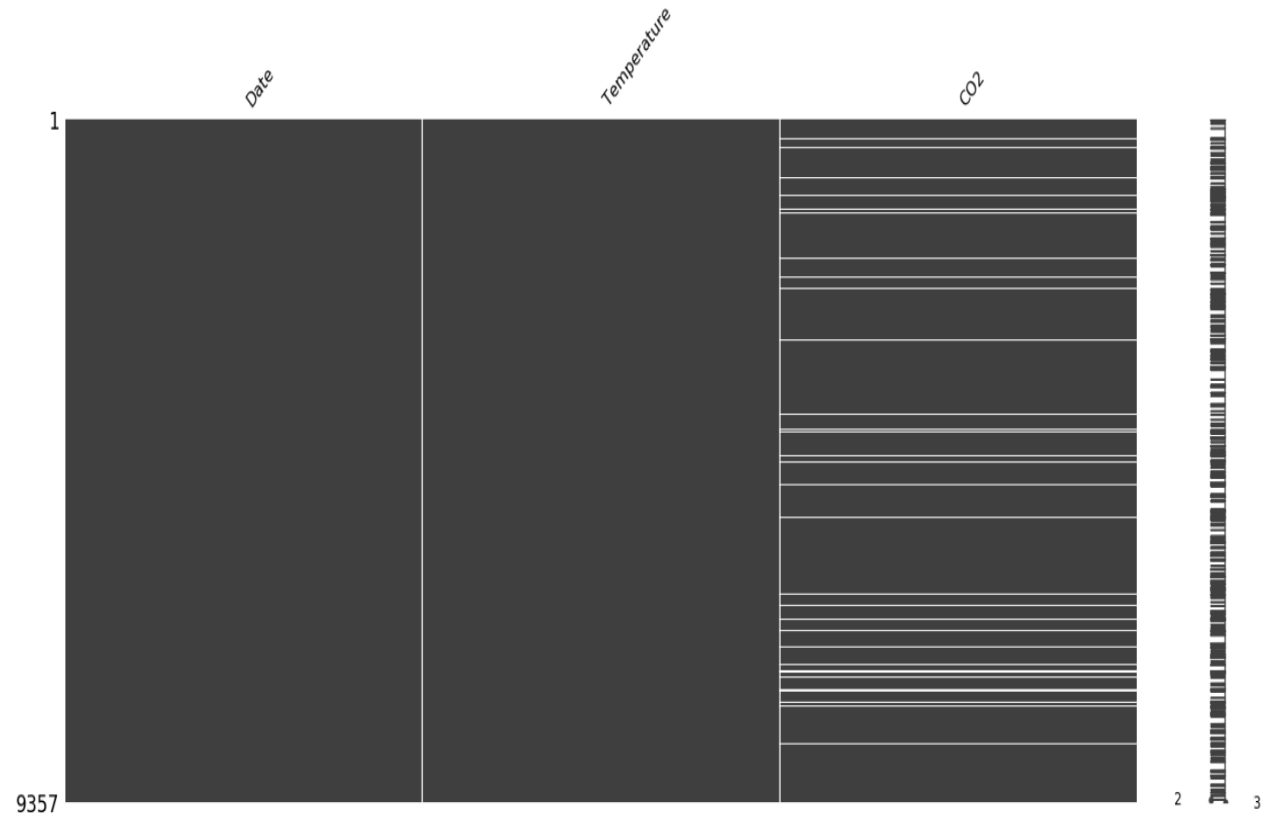
Date	0
Temperature	0
C02	366
dtype:	int64

Problemas Avanzados de Datos – Integridad (Compleitud)

Ejemplo: Calidad del aire. Visualización de datos nulos

```
import missingno as msno
import matplotlib.pyplot as plt
# Visualize missingness
msno.matrix(airquality)
plt.show()
```

pip install missingno 



Problemas Avanzados de Datos – Integridad (Completitud)

Ejemplo: Calidad del aire. Análisis de datos nulos

```
# Isolate missing and complete values aside  
missing = airquality[airquality['CO2'].isna()]  
complete = airquality[~airquality['CO2'].isna()]
```



```
# Describe missing DataFramee  
missing.describe()
```

	Temperature	CO2
count	366.000000	0.0
mean	-39.655738	NaN
std	5.988716	NaN
min	-49.000000	NaN
...	...	...
max	-30.000000	NaN

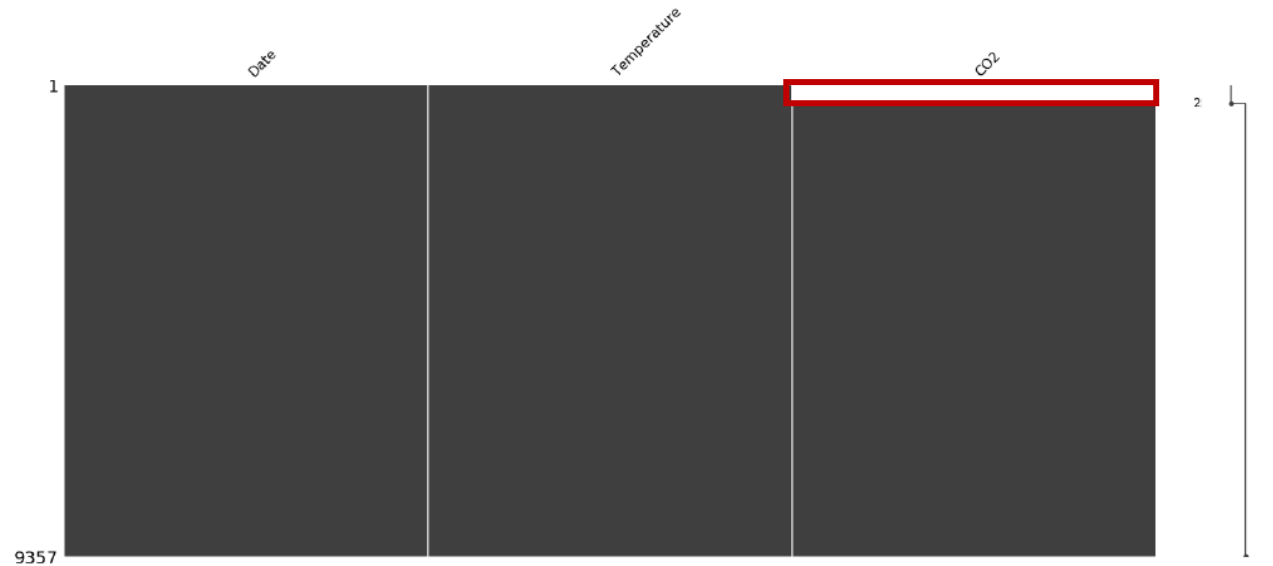
```
# Describe complete DataFramee  
complete.describe()
```

	Temperature	CO2
count	8991.000000	8991.000000
mean	18.317829	1.739584
std	8.832116	1.537580
min	-1.900000	0.000000
...	...	...
max	44.600000	11.900000

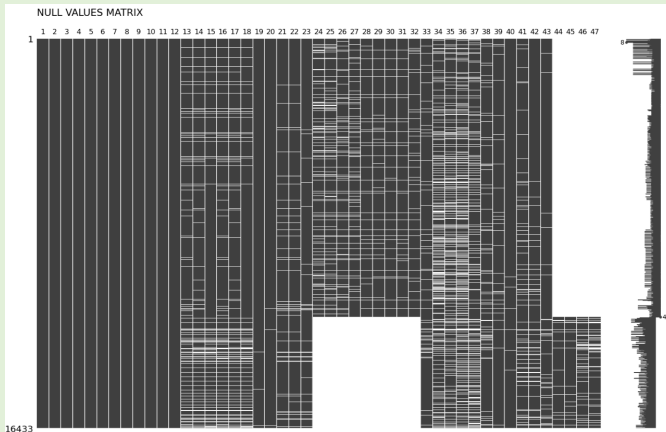
Problemas Avanzados de Datos – Integridad (Compleitud)

Ejemplo: Calidad del aire. Visualización de datos nulos

```
sorted_airquality = airquality.sort_values(by = 'Temperature')  
msno.matrix(sorted_airquality)  
plt.show()
```

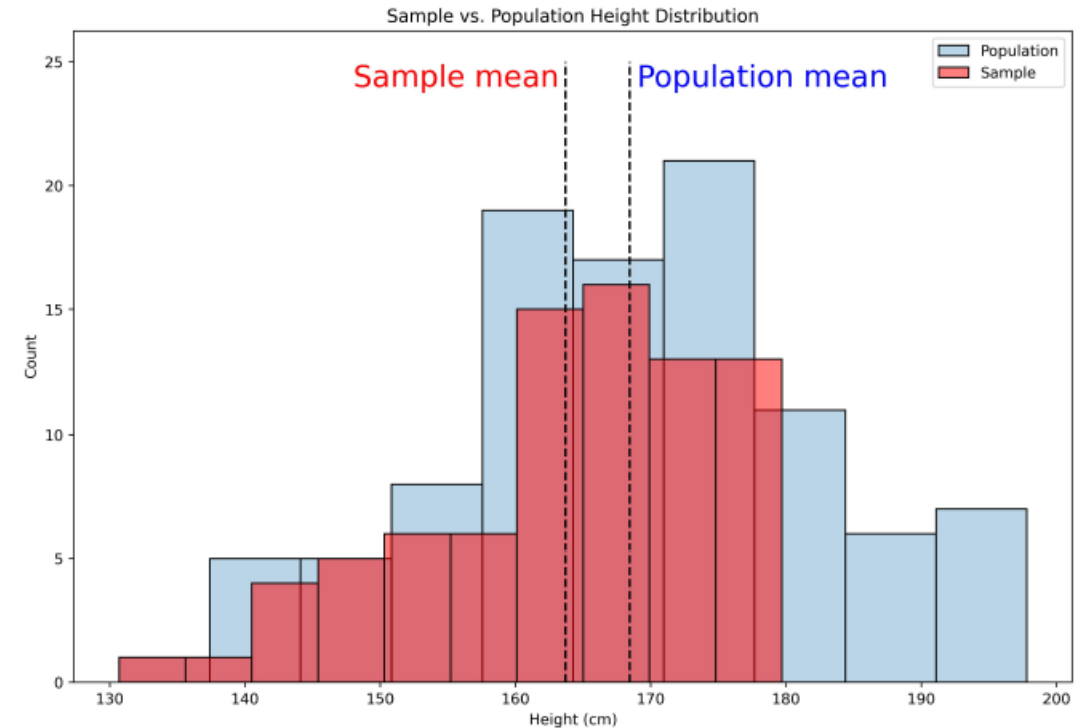


Tratamiento de Datos Nulos



¿Por qué es un problema que falten datos?

- Afectan distribuciones.
 - Faltan las estaturas de los estudiantes más altos.
- Muestra menos representativa de la población.
 - Ciertos grupos están representados de forma desproporcionada. Ejemplo: Falta de datos de los estudiantes de mayor edad.
- Pueden dar lugar a conclusiones erróneas.



Tipos de Datos Nulos



*Missing Completely
at Random*

(MCAR)

Los datos nulos son completamente aleatorios y no dependen de ningún dato observado o no observado.

Ejemplo: Falla de sensores.

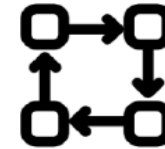


*Missing at
Random*

(MAR)

Los datos nulos están relacionados con los **datos observados**.

Ejemplo: En un estudio sobre ingresos y nivel de satisfacción laboral, las personas con mayor satisfacción laboral pueden ser menos propensas a reportar sus ingresos.



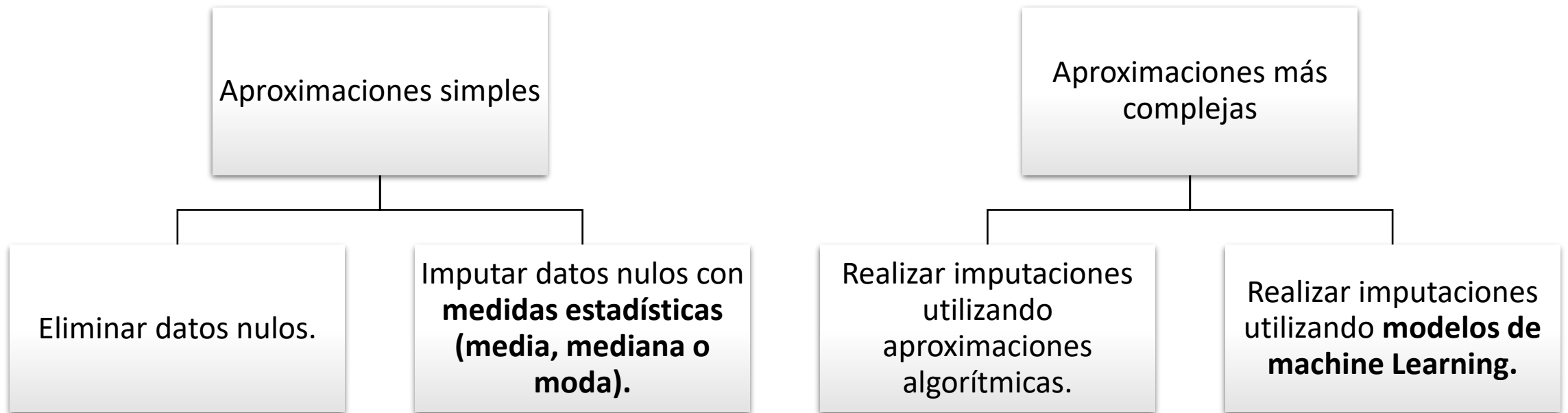
*Missing Not at
Random*

(MNAR)

Los datos nulos están relacionados con los **datos no observados**.

Ejemplo: Valores nulos de temperaturas debido a altos valores de este parámetro.

¿Cómo tratar los datos nulos?



Aproximaciones Simples

Ejemplo de calidad del aire

```
airquality.head()
```

	Date	Temperature	C02
0	05/03/2005	8.5	2.5
1	23/08/2004	21.8	0.0
2	18/02/2005	6.3	1.0
3	08/02/2005	-31.0	NaN
4	13/03/2005	19.9	0.1

1. Eliminación de datos nulos

```
# Drop missing values
```

```
airquality_dropped = airquality.dropna(subset = ['C02'])  
airquality_dropped.head()
```

	Date	Temperature	C02
0	05/03/2005	8.5	2.5
1	23/08/2004	21.8	0.0
2	18/02/2005	6.3	1.0
4	13/03/2005	19.9	0.1
5	02/04/2005	17.0	0.8

Aproximaciones Simples

Ejemplo de calidad del aire

```
airquality.head()
```

	Date	Temperature	C02
0	05/03/2005	8.5	2.5
1	23/08/2004	21.8	0.0
2	18/02/2005	6.3	1.0
3	08/02/2005	-31.0	NaN
4	13/03/2005	19.9	0.1

2. Imputación con medidas estadísticas

```
co2_mean = airquality['C02'].mean()  
airquality_imputed = airquality.fillna({'C02': co2_mean})  
airquality_imputed.head()
```

	Date	Temperature	C02
0	05/03/2005	8.5	2.500000
1	23/08/2004	21.8	0.000000
2	18/02/2005	6.3	1.000000
3	08/02/2005	-31.0	1.739584
4	13/03/2005	19.9	0.100000

Ejemplo - Empleos en Ciencia de Datos

Column	Description	Data type
Working_Year	Year the data was obtained	Float
Designation	Job title	String
Experience	Experience level e.g., "Mid" , "Senior"	String
Employment_Status	Type of employment contract e.g., "FT" , "PT"	String
Employee_Location	Country of employment	String
Company_Size	Labels for company size e.g., "S" , "M" , "L"	String
Remote_Working_Ratio	Percentage of time working remotely	Integer
Salary_USD	Salary in US dollars	Float

EDA – Revisión de Datos Nulos

```
print(salaries.isna().sum())
```

```
Working_Year      12  
Designation       27  
Experience        33  
Employment_Status 31  
Employee_Location 28  
Company_Size      40  
Remote_Working_Ratio 24  
Salary_USD        60  
dtype: int64
```


Estrategias para Manejar Datos Nulos

1. Eliminar datos nulos.

- 5% o menos del total de datos.

2. Imputar datos nulos utilizando media, mediana o moda.

- Depende de la distribución y el contexto.
- Mediana maneja mejor los outliers.

3. Imputar datos nulos por subgrupos.

- Diferentes niveles de experiencia tienen diferentes medianas salariales.

Estrategias para Manejar Datos Nulos

1. Eliminar Datos Nulos

```
threshold = len(salaries) * 0.05  
print(threshold)
```

```
30
```

```
cols_to_drop = salaries.columns[salaries.isna().sum() <= threshold]  
print(cols_to_drop)
```

```
Index(['Working_Year', 'Designation', 'Employee_Location',  
      'Remote_Working_Ratio'],  
      dtype='object')
```

```
salaries.dropna(subset=cols_to_drop, inplace=True)
```

Estrategias para Manejar Datos Nulos

2. Imputar Datos Nulos con Métricas Estadísticas

```
cols_with_missing_values = salaries.columns[salaries.isna().sum() > 0]
print(cols_with_missing_values)
```

```
Index(['Experience', 'Employment_Status', 'Company_Size', 'Salary_USD'],
      dtype='object')
```

```
for col in cols_with_missing_values[:-1]:
    salaries[col].fillna(salaries[col].mode()[0])
```

Estrategias para Manejar Datos Nulos

Revisar columnas que aún mantienen datos nulos

```
print(salaries.isna().sum())
```

Working_Year	0
Designation	0
Experience	0
Employment_Status	0
Employee_Location	0
Company_Size	0
Remote_Working_Ratio	0
Salary_USD	41

Estrategias para Manejar Datos Nulos

3. Imputar Datos Nulos por Subgrupos

```
salaries_dict = salaries.groupby("Experience")["Salary_USD"].median().to_dict()
print(salaries_dict)
```

```
{'Entry': 55380.0, 'Executive': 135439.0, 'Mid': 74173.5, 'Senior': 128903.0}
```

```
salaries["Salary_USD"] = salaries["Salary_USD"].fillna(salaries["Experience"].map(salaries_dict))
```

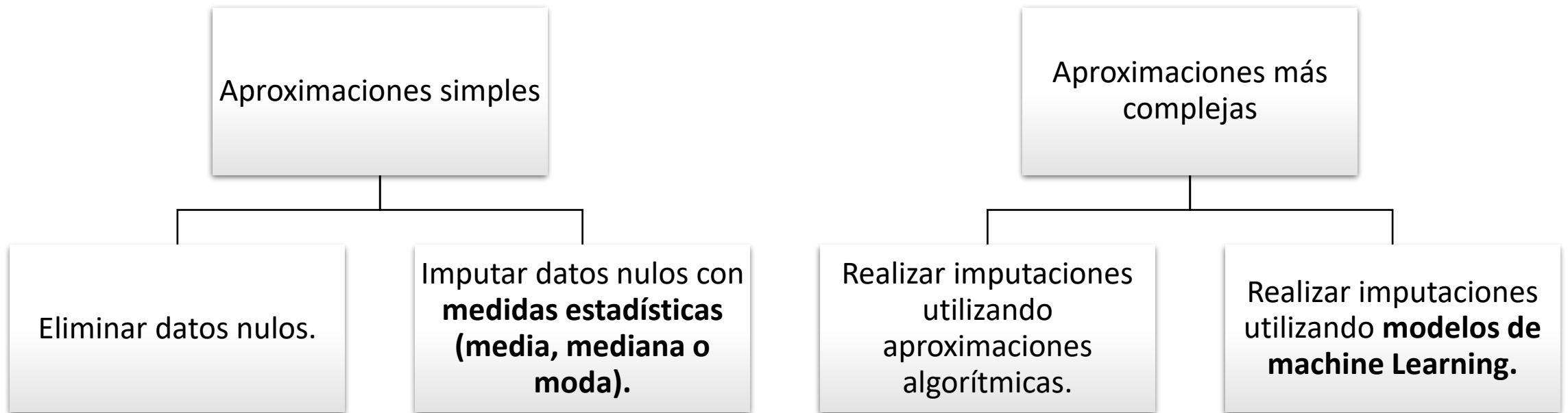
Estrategias para Manejar Datos Nulos

¡Comprobar que ya no hay Datos Nulos!

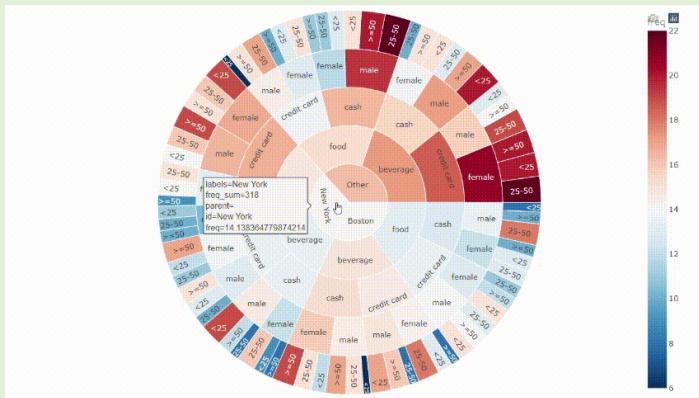
```
print(salaries.isna().sum())
```

```
Working_Year      0  
Designation      0  
Experience        0  
Employment_Status 0  
Employee_Location 0  
Company_Size      0  
Remote_Working_Ratio 0  
Salary_USD        0  
dtype: int64
```

¿Cómo tratar los datos nulos?



Tratamiento de Outliers



¿Qué es un dato atípico?

- **Observaciones alejadas de la distribución general de datos.**
 - ✓ Precio promedio de una casa: \$400,000.
 - ✓ Dato atípico del precio de una casa: \$5,000,000.
- **Debería analizarse el motivo de la diferencia de estos valores:**
 - ✓ Ubicación.
 - ✓ Número de cuartos.
 - ✓ Tamaño



¿Cómo Identificar Outliers?

Métricas Estadísticas

```
print(salaries["Salary_USD"].describe())
```

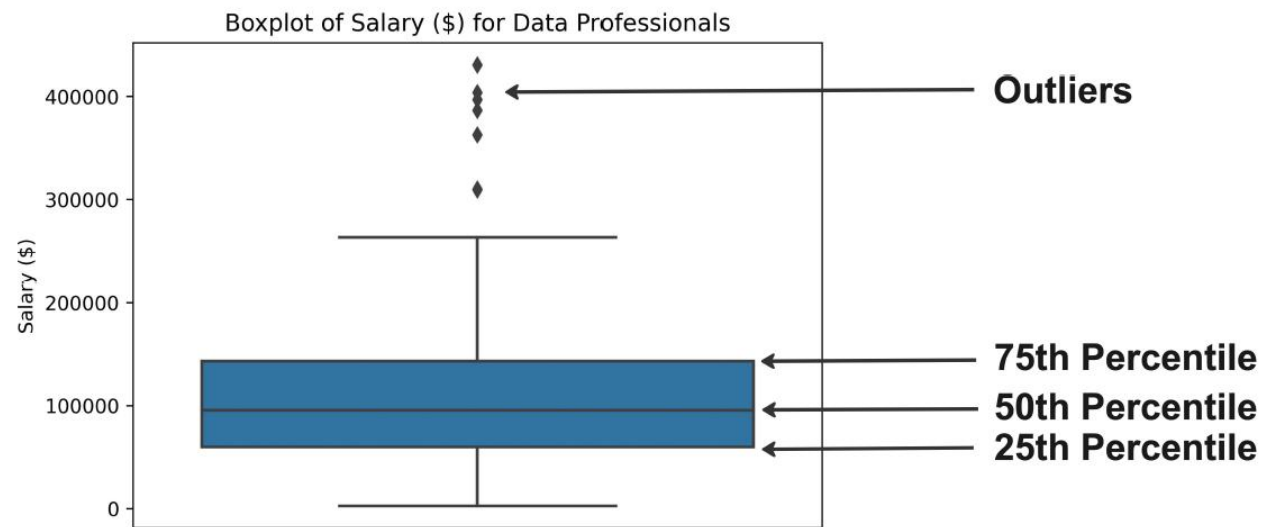
```
count      518.000  
mean    104905.826  
std     62660.107  
min      3819.000  
25%     61191.000  
50%     95483.000  
75%    137496.000  
max     429675.000  
Name: Salary_USD, dtype: float64
```

¿Cómo Identificar Outliers?

Rango Intercuartil (IQR)

- $IQR = 75th - 25th$

```
sns.boxplot(data=salaries,  
            y="Salary_USD")  
plt.show()
```



Tratamiento de Outliers

Rango Intercuartil (IQR)

- $IQR = 75th - 25th$
- **Outliers superiores** $> 75th + (1.5 * IQR)$
- **Outliers inferiores** $< 25th - (1.5 * IQR)$

```
# 75th percentile
seventy_fifth = salaries["Salary_USD"].quantile(0.75)

# 25th percentile
twenty_fifth = salaries["Salary_USD"].quantile(0.25)

# Interquartile range
salaries_iqr = seventy_fifth - twenty_fifth

print(salaries_iqr)
```

```
76305.0
```

Tratamiento de Outliers

Rango Intercuartil (IQR) – Límites Superiores e Inferiores

```
# Upper threshold  
upper = seventy_fifth + (1.5 * salaries_iqr)  
  
# Lower threshold  
lower = twenty_fifth - (1.5 * salaries_iqr)  
  
print(upper, lower)
```

```
251953.5 -53266.5
```

Tratamiento de Outliers

Rango Intercuartil (IQR) – Identificar Observaciones Atípicas

```
salaries[(salaries["Salary_USD"] < lower) | (salaries["Salary_USD"] > upper)] \
[["Experience", "Employee_Location", "Salary_USD"]]
```

	Experience	Employee_Location	Salary_USD
29	Mid	US	429675.0
67	Mid	US	257805.0
80	Senior	US	263534.0
83	Mid	US	429675.0
133	Mid	US	403895.0
410	Executive	US	309366.0
441	Senior	US	362837.0
445	Senior	US	386708.0
454	Senior	US	254368.0

Tratamiento de Outliers

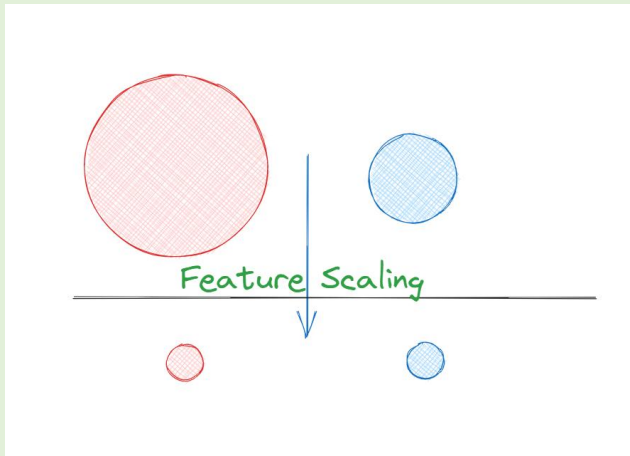
Rango Intercuartil (IQR) – Filtrar Outliers

```
no_outliers = salaries[(salaries["Salary_USD"] > lower) & (salaries["Salary_USD"] < upper)]
```

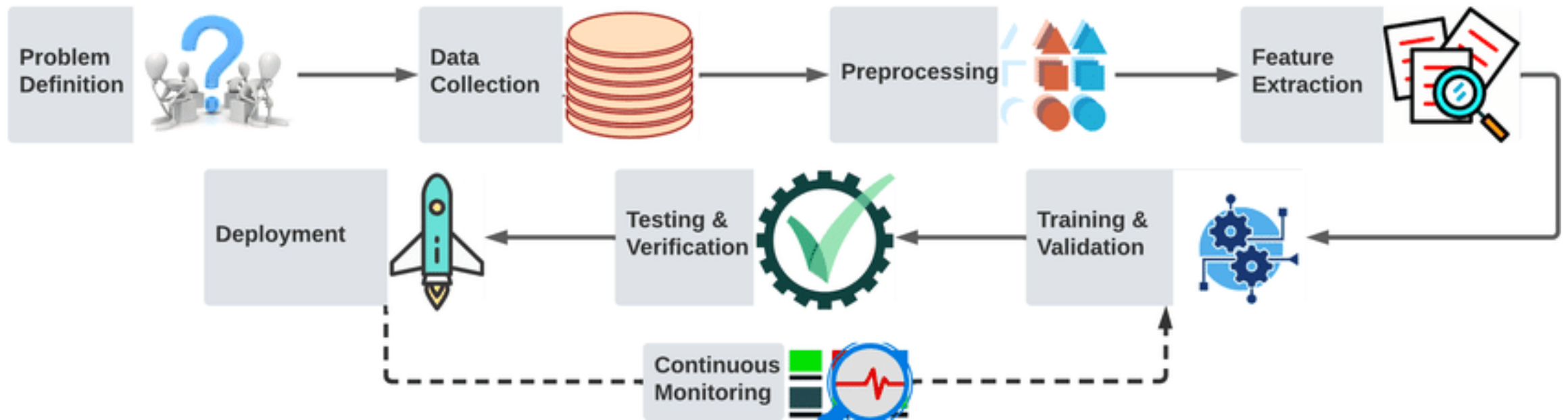
```
print(no_outliers["Salary_USD"].describe())
```

```
count      509.000000
mean      100674.567780
std       53643.050057
min       3819.000000
25%       60928.000000
50%       95483.000000
75%      134059.000000
max      248257.000000
Name: Salary_USD, dtype: float64
```

Escalado y Normalización de Características



Machine Learning Pipeline



¿Qué es Estandarización / Normalización de Datos?

- **Normalización:** transformar los datos continuos para que estén distribuidos normalmente.
 - ✓ los modelos de la librería **scikit-learn** asumen datos distribuidos normalmente.
 - ✓ El uso de datos de entrenamiento no normalizados puede introducir sesgos.
 - ✓ **Tipos de normalización de datos:**
 - ☐ Normalización logarítmica.
 - ☐ Escalado de características.
 - ✓ Aplicado a datos numéricos continuos.

¿Cuándo Normalizar Datos?

Modelos que utilizan distancias lineales

➤ Ejemplos:

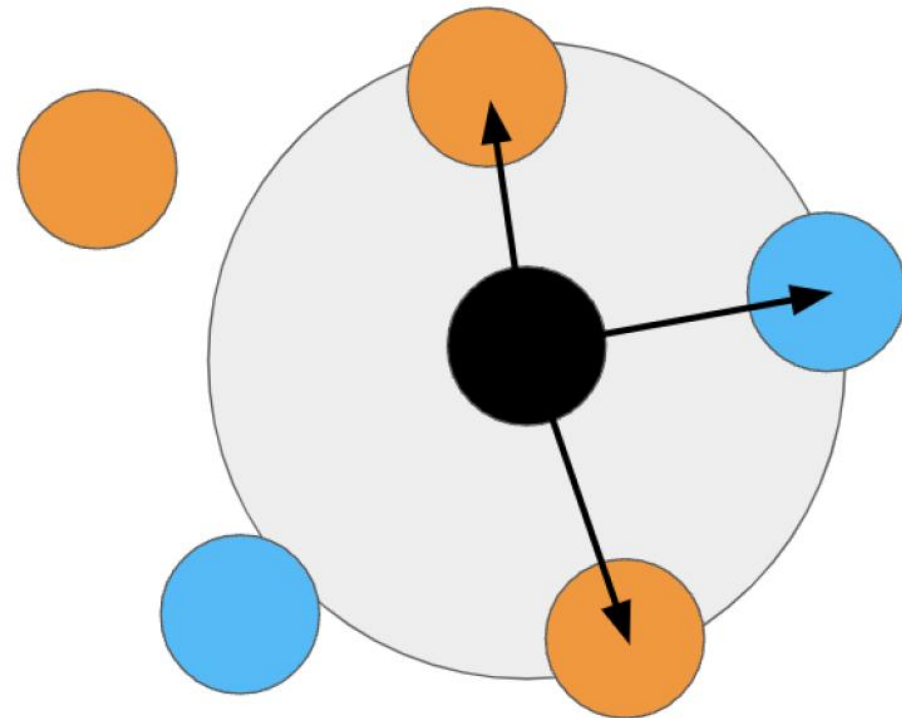
- ✓ K-Nearest Neighbors (KNN).
- ✓ Regresión Lineal.
- ✓ K-Means Clustering.

Alta Varianza

- Variables de los conjuntos de datos suelen tener alta varianza.

Diferentes Escalas

- Características están en diferentes escalas.
- **Ejemplo:** Predecir precios de casas utilizando número de cuartos y precio de la última venta.



Tipos de Normalización de Datos

➤ *Normalización logarítmica:*

- ✓ Recomendada para características que tengan alta varianza.
- ✓ Se aplica transformación logarítmica.
- ✓ Aplica logaritmo natural, utilizando la constante $e = 2.718$.
- ✓ $e^{3.4} = 30$

Número	Log
30	3.4
300	5.7
3000	8

Tipos de Normalización de Datos

➤ Normalización logarítmica:

```
print(df)
```

	col1	col2
0	1.00	3.0
1	1.20	45.5
2	0.75	28.0
3	1.60	100.0

```
print(df.var())
```

col1	0.128958
col2	1691.729167

dtype: float64

```
import numpy as np
df["log_2"] = np.log(df["col2"])
print(df)
```

	col1	col2	log_2
0	1.00	3.0	1.098612
1	1.20	45.5	3.817712
2	0.75	28.0	3.332205
3	1.60	100.0	4.605170

```
print(df[["col1", "log_2"]].var())
```

col1	0.128958
log_2	2.262886

dtype: float64

Tipos de Normalización de Datos

➤ *Escalado de Características (Feature Scaling):*

- ✓ Características en diferentes escalas.
- ✓ Modelos con características lineales.
- ✓ Centra características alrededor del cero y transforma la varianza a 1.
- ✓ Transforma las características a una distribución aproximadamente normal.

Tipos de Normalización de Datos

➤ Escalado de Características (Feature Scaling):

```
print(df)
```

	col1	col2	col3
0	1.00	48.0	100.0
1	1.20	45.5	101.3
2	0.75	46.2	103.5
3	1.60	50.0	104.0

```
print(df.var())
```

col1	0.128958
col2	4.055833
col3	3.526667

dtype: float64

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler()  
df_scaled = pd.DataFrame(scaler.fit_transform(df),  
                          columns=df.columns)
```

```
print(df_scaled)
```

	col1	col2	col3
0	-0.442127	0.329683	-1.352726
1	0.200967	-1.103723	-0.553388
2	-1.245995	-0.702369	0.799338
3	1.487156	1.476409	1.106776



Librería scikit-learn

conda install sklearn

```
print(df_scaled.var())
```

col1	1.333333
col2	1.333333
col3	1.333333

dtype: float64

Datos y Modelos Normalizados

Ejemplo: Algoritmo K Nearest Neighbors.

```
from sklearn.model_selection import train_test_split  
from sklearn.neighbors import KNeighborsClassifier
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, random_state=42)
```

```
knn = KNeighborsClassifier()  
scaler = StandardScaler()
```

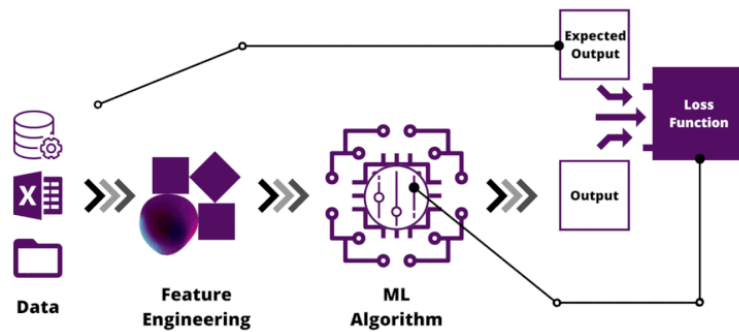
```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
knn.fit(X_train_scaled, y_train)
```

```
knn.score(X_test_scaled, y_test)
```


Ingeniería de Características



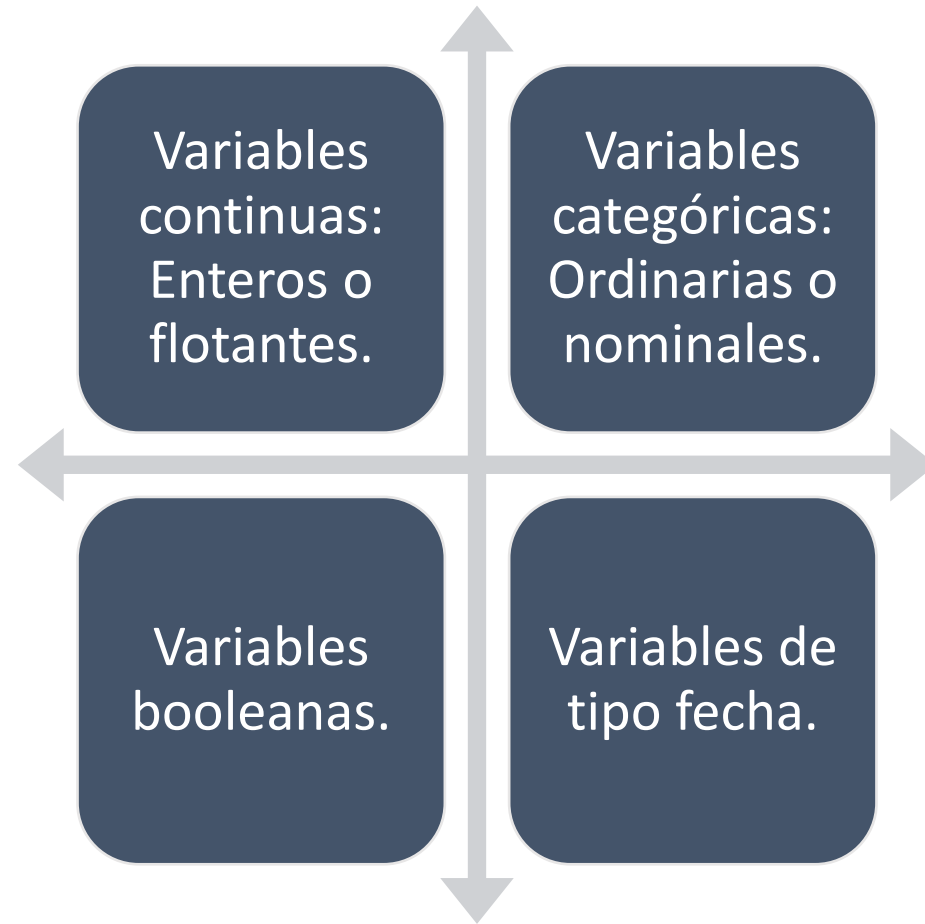
¿Qué es la Ingeniería de Características?

Feature Engineering:

Creación de **nuevas características** a partir de las variables existentes.

- Mejora rendimiento del modelo de datos.
- Conocimiento de las relaciones entre las características.
- ¡Primero hay que entender los datos!
- Depende en gran medida del conjunto de datos.

Feature Engineering - Diferentes Tipos de Datos



Feature Engineering: Escenarios

Id	Text
1	"Feature engineering is fun!"
2	"Feature engineering is a lot of work."
3	"I don't mind feature engineering."

user	fav_color
1	blue
2	green
3	orange

Id	Date
4	July 30 2011
5	January 29 2011
6	February 05 2011

user	test1	test2	test3
1	90.5	89.6	91.4
2	65.5	70.6	67.3
3	78.1	80.7	81.8

Codificación de Variables Categóricas - Pandas

Variables Binarias

	user	subscribed	fav_color
0	1	y	blue
1	2	n	green
2	3	n	orange
3	4	y	green

```
print(users["subscribed"])
```

```
0    y
1    n
2    n
3    y
Name: subscribed, dtype: object
```

```
print(users[["subscribed", "sub_enc"]])
```

```
subscribed  sub_enc
0           y       1
1           n       0
2           n       0
3           y       1
```

```
users["sub_enc"] = users["subscribed"].apply(lambda val: 1 if val == "y" else 0)
```

Codificación de Variables Categóricas – scikit-learn

Variables Binarias

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()  
users["sub_enc_le"] = le.fit_transform(users["subscribed"])  
  
print(users[["subscribed", "sub_enc_le"]])
```

	subscribed	sub_enc_le
0	y	1
1	n	0
2	n	0
3	y	1

Codificación de Variables Categóricas

One-Hot Encoding: Válido para variables categóricas nominales.

fav_color
blue
green
orange
green

Values: [blue, green, orange]

fav_color_enc
[1, 0, 0]
[0, 1, 0]
[0, 0, 1]
[0, 1, 0]

- blue: [1, 0, 0]
- green: [0, 1, 0]
- orange: [0, 0, 1]

Codificación de Variables Categóricas

```
print(users["fav_color"])
```

```
0    blue
1   green
2  orange
3   green
Name: fav_color, dtype: object
```

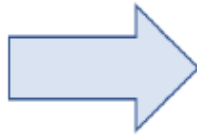
```
print(pd.get_dummies(users["fav_color"]))
```

	blue	green	orange
0	1	0	0
1	0	1	0
2	0	0	1
3	0	1	0

Codificación de Variables Categóricas

One-Hot Encoding: Válido para variables categóricas nominales.

Index	Country
1	'India'
2	'USA'
3	'UK'
4	'UK'
5	'France'
...	...



Index	C_India	C_USA	C_UK	C_France
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0
4	0	0	1	0
5	0	0	0	1
...	...	...	...	...

Codificación de Variables Categóricas

One-Hot Encoding: Válido para variables categóricas nominales.

```
pd.get_dummies(df, columns=['Country'],  
               prefix='C')
```

	C_France	C_India	C_UK	C_USA
0	0	1	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	0
4	1	0	0	0

```
pd.get_dummies(df, columns=['Country'],  
               drop_first=True, prefix='C')
```

	C_India	C_UK	C_USA
0	1	0	0
1	0	0	1
2	0	1	0
3	0	1	0
4	0	0	0

Codificación de Variables Categóricas

Reducir Categorías

```
counts = df['Country'].value_counts()  
print(counts)
```

```
'USA'      8  
'UK'       6  
'India'    2  
'France'   1  
Name: Country, dtype: object
```

```
mask = df['Country'].isin(counts[counts < 5].index)  
df['Country'][mask] = 'Other'  
print(pd.value_counts(colors))
```

```
'USA'      8  
'UK'       6  
'Other'    3  
Name: Country, dtype: object
```

Feature Engineering: Variables de Tipo Fecha

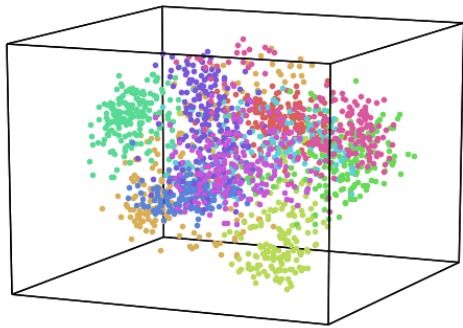
```
print(purchases)
```

		date	purchase
0	July 30	2011	\$45.08
1	February 01	2011	\$19.48
2	January 29	2011	\$76.09
3	March 31	2012	\$32.61
4	February 05	2011	\$75.98

```
purchases["date_converted"] = pd.to_datetime(purchases["date"])
purchases['month'] = purchases["date_converted"].dt.month
print(purchases)
```

		date	purchase	date_converted	month
0	July 30	2011	\$45.08	2011-07-30	7
1	February 01	2011	\$19.48	2011-02-01	2
2	January 29	2011	\$76.09	2011-01-29	1
3	March 31	2012	\$32.61	2012-03-31	3
4	February 05	2011	\$75.98	2011-02-05	2

Selección de Características



Selección de Características

¿Qué es la selección de características?

- Seleccionar las variables que serán utilizadas en el modelo de datos.
- No crear nuevas variables.
- Mejorar el rendimiento del modelo.

Cuando se seleccionan características:

city	state	lat	long
hico	tx	31.982778	-98.033333
mackinaw city	mi	45.783889	-84.727778
winchester	ky	37.990000	-84.179722

- Reducir ruido.
- Características fuertemente correlacionadas con la variable dependiente.
- Reducir varianza general.

Selección de Características – Características Redundantes

- Remover características fuertemente correlacionadas entre sí.
- Remover características duplicadas.

¿Cuáles características seleccionaría?

city	state	lat	long
hico	tx	31.982778	-98.033333
mackinaw city	mi	45.783889	-84.727778
winchester	ky	37.990000	-84.179722

Selección de Características – Características Redundantes

Características Correlacionadas

- Modelos lineales asumen independencia de las características.
- Utilizar coeficiente de correlación de Pearson.
- Utilizar coeficiente de Spearman (tendencias no lin)

```
print(df)
```

	A	B	C
0	3.06	3.92	1.04
1	2.76	3.40	1.05
2	3.24	3.17	1.03
...			

```
print(df.corr())
```

	A	B	C
A	1.000000	0.787194	0.543479
B	0.787194	1.000000	0.565468
C	0.543479	0.565468	1.000000

Referencias

- *Unpingco, J. (2021). Python Programming for Data Analysis. Springer*
- *Zelle, J. M. (2010). Python Programming: An Introduction to Computer Science. Franklin, Beedle & Associates Inc.*
- *Datacamp Courses*

Créditos: Freddy Paul Carrión Maldonado